COMSATS University Islamabad

Attock Campus

Department Of Computer Science

Course	***INFORMATION SECURITY LAB***
Instructor	***Ms. Ambreen Gul***
Assignment No.	***03***

Student Details

Registration No.	***Name***
SP24-BSE-045	***SHEIKH FARAZ***

Task 1: Basic Blockchain Implementation [5]

Objective: Understand the core structure of blockchain Tasks:

•Create a simple blockchain class in Python

• Each block should contain:

o Index

o Timestamp

o Data

o Previous Hash

o Hash (calculated using SHA256)

• Implement functions for:

o Creating a new block

o Linking blocks using hashes

• Add at least 4–5 blocks and display the chain

• Demonstrate how modifying one block affects the entire chain

CODE:

```
import hashlib
import datetime

# Simple Block class
class Block:  2 usages
    def __init__(self, index, data, previous_hash):
        self.index = index                    # block number
        self.timestamp = datetime.datetime.now()  # time of block creation
        self.data = data                      # block data
        self.previous_hash = previous_hash    # link to previous block
        self.hash = self.create_hash()        # current block hash

    # function to create hash
    def create_hash(self):  3 usages (2 dynamic)
        text = str(self.index) + str(self.timestamp) + self.data + self.previous_hash
        return hashlib.sha256(text.encode()).hexdigest()

# Blockchain class
class Blockchain:  1 usage
    def __init__(self):
        self.chain = []               # list to store blocks
        self.create_first_block()     # create first block

    # create first block (Genesis Block)
    def create_first_block(self):  1 usage
```

```python
        block = Block( index: 0,  data: "First Block",  previous_hash: "0")
        self.chain.append(block)

    # add new block
    def add_block(self, data):  4 usages
        last_block = self.chain[-1]   # get last block
        new_block = Block(len(self.chain), data, last_block.hash)
        self.chain.append(new_block)

    # display blockchain
    def show(self):  2 usages
        for block in self.chain:
            print("Index:", block.index)
            print("Data:", block.data)
            print("Hash:", block.hash)
            print("Previous Hash:", block.previous_hash)
            print("-----------")

# create blockchain
bc = Blockchain()

# add blocks
bc.add_block("Block 1")
bc.add_block("Block 2")
```

```python
bc.add_block("Block 3")
bc.add_block("Block 4")

print("Before changing data:")
bc.show()

# change data (tampering)
bc.chain[1].data = "Hacked Block"
bc.chain[1].hash = bc.chain[1].create_hash()

print("\nAfter changing data:")
bc.show()
```

OUTPUT:

```
C:\Users\hashi\PycharmProjects\PythonProject\.venv\Scripts\python.exe "C:\Users\hashi\Desktop\un
Before changing data:
Index: 0
Data: First Block
Hash: f21dfa86a4f6a72bba3f9ad762d19d2719e961fe78fd2fb3eee1bbbf70eeb9c3
Previous Hash: 0
-----------
Index: 1
Data: Block 1
Hash: e0670cb4c03adf69c9def51e982ee9622d93029b178173f4c85d051856ee24ab
Previous Hash: f21dfa86a4f6a72bba3f9ad762d19d2719e961fe78fd2fb3eee1bbbf70eeb9c3
-----------
Index: 2
Data: Block 2
Hash: 44c92d2daaf1e0b0fef8f6016c682af89d27979c9e381807aab2ed3ce12e2a83
Previous Hash: e0670cb4c03adf69c9def51e982ee9622d93029b178173f4c85d051856ee24ab
-----------
Index: 3
Data: Block 3
Hash: 97df566d3323579da6f243192b3d2e8dd98b61c636c0e7ed9857a1e0c72ca44e
Previous Hash: 44c92d2daaf1e0b0fef8f6016c682af89d27979c9e381807aab2ed3ce12e2a83
```

```
Index: 4
Data: Block 4
Hash: 18e9f00cb7f2258e3ebf33393d577fb3c2511872a036ef6a8e8037a137c1b6de
Previous Hash: 97df566d3323579da6f243192b3d2e8dd98b61c636c0e7ed9857a1e0c72ca44e
-----------

After changing data:
Index: 0
Data: First Block
Hash: f21dfa86a4f6a72bba3f9ad762d19d2719e961fe78fd2fb3eee1bbbf70eeb9c3
Previous Hash: 0
-----------
Index: 1
Data: Hacked Block
Hash: 7a6a4602088df44b4a46e1e2847498af823dd5776a2580f6c38f67eb0c38f2f3
Previous Hash: f21dfa86a4f6a72bba3f9ad762d19d2719e961fe78fd2fb3eee1bbbf70eeb9c3
-----------
```

```
Index: 2
Data: Block 2
Hash: 44c92d2daaf1e0b0fef8f6016c682af89d27979c9e381807aab2ed3ce12e2a83
Previous Hash: e0670cb4c03adf69c9def51e982ee9622d93029b178173f4c85d051856ee24ab
-----------
Index: 3
Data: Block 3
Hash: 97df566d3323579da6f243192b3d2e8dd98b61c636c0e7ed9857a1e0c72ca44e
Previous Hash: 44c92d2daaf1e0b0fef8f6016c682af89d27979c9e381807aab2ed3ce12e2a83
-----------
Index: 4
Data: Block 4
Hash: 18e9f00cb7f2258e3ebf33393d577fb3c2511872a036ef6a8e8037a137c1b6de
Previous Hash: 97df566d3323579da6f243192b3d2e8dd98b61c636c0e7ed9857a1e0c72ca44e
-----------

Process finished with exit code 0
```

LINE BY LINE EXPLANATION:

import hashlib

This imports a library used to create hash values (SHA-256)

import datetime

This is used to get current date and time

class Block:

We create a Block class (each block in blockchain)

def __init__(self, index, data, previous_hash):

Constructor function (runs automatically when block is created)

self.index = index

Stores block number (0,1,2,...)

self.timestamp = datetime.datetime.now()

Stores current time when block is created

self.data = data

Stores data inside the block

self.previous_hash = previous_hash

Stores hash of previous block (this links blocks)

self.hash = self.create_hash()

Creates hash for current block

def create_hash(self):

Function to generate hash

text = str(self.index) + str(self.timestamp) + self.data + self.previous_hash

Combines all block data into one string

return hashlib.sha256(text.encode()).hexdigest()

Converts text into SHA-256 hash (unique code)

class Blockchain:

Create Blockchain class (collection of blocks)

def __init__(self):

Constructor for blockchain

self.chain = []

Empty list to store blocks

self.create_first_block()

Calls function to create first block

def create_first_block(self):

Function to create first block

block = Block(0, "First Block", "0")

Creates Genesis Block (first block)

self.chain.append(block)

Adds block to chain

def add_block(self, data):

Function to add new block

last_block = self.chain[-1]

Gets last block from chain

new_block = Block(len(self.chain), data, last_block.hash)

Creates new block
Uses previous block's hash

self.chain.append(new_block)

Adds new block to chain

def show(self):

Function to display blockchain

for block in self.chain:

Loop through all blocks

print("Index:", block.index)

Print block number

print("Data:", block.data)

Print data

print("Hash:", block.hash)

Print current hash

print("Previous Hash:", block.previous_hash)

Print previous hash

bc = Blockchain()

Create blockchain object

bc.add_block("Block 1")

Add block 1

(similar for others)

bc.show()

Display blockchain

bc.chain[1].data = "Hacked Block"

Change data (tampering)

bc.chain[1].hash = bc.chain[1].create_hash()

Recalculate hash

Now chain becomes invalid

SECURITY ANALYSIS

1. Blockchain Security Analysis

Data Integrity

- Each block has a hash value
- If data is changed → hash also changes
  This makes tampering easy to detect

Linking of Blocks

- Each block stores previous hash
- This creates a chain

If one block changes → all next blocks become invalid

Immutability

- Once block is added, it should not be changed
- Any change breaks the chain

Provides strong protection against modification

Tampering Detection

- When we changed:

bc.chain[1].data = "Hacked Block"

Hash changed
But next blocks still have old hash

Result:

- Chain becomes inconsistent
- Attack is detected easily

Weakness

- This is a basic blockchain
- No:
    - Mining
    - Consensus algorithm
    - Network security

So it is not secure like real blockchain

Task 2: Elliptic Curve Cryptography (ECC) Basics [5]

Objective: Understand ECC key generation and encryption

Tasks:

- **Implement basic ECC operations using small curve parameters:**

o Point addition

o Point doubling

- **Generate public and private keys manually**

- **Perform a simple encryption and decryption using ECC concept (e.g., EC-ElGamal simplified version)**

- **Test using a small numeric message**

CODE:

```
# Simple ECC (Fixed Version)

p = 17   # prime number (modulus)
a = 2
b = 2

# Point addition
def add(P, Q):
    # If one point is None, return the other
    if P is None:
        return Q
    if Q is None:
        return P

    # If points are same → use doubling
    if P == Q:
        return double(P)

    x1, y1 = P
    x2, y2 = Q

    # If x1 == x2 → vertical line (no inverse possible)
    if (x2 - x1) % p == 0:
        return None   # represents point at infinity
```

```python
    # slope calculation
    m = ((y2 - y1) * pow(x2 - x1, -1, p)) % p

    # new point
    x3 = (m*m - x1 - x2) % p
    y3 = (m*(x1 - x3) - y1) % p

    return (x3, y3)


# Point doubling
def double(P):
    if P is None:
        return None

    x, y = P

    # If y == 0 → no inverse possible
    if y % p == 0:
        return None

    # slope calculation
    m = ((3*x*x + a) * pow(2*y, -1, p)) % p
```

```python
    # new point
    x3 = (m*m - 2*x) % p
    y3 = (m*(x - x3) - y) % p

    return (x3, y3)


# Scalar multiplication (k * P)
def multiply(k, P):
    result = None   # start with point at infinity

    for i in range(k):
        result = add(result, P)

    return result


# Base point (valid point on curve)
G = (5, 1)

# Private key (secret)
private_key = 2

# Public key = private_key * G
```

```python
public_key = multiply(private_key, G)

print("Private Key:", private_key)
print("Public Key:", public_key)


# Simple message
message = 5

# Encryption
k = 2   # random number

C1 = multiply(k, G)                     # first part
C2 = message + multiply(k, public_key)[0]   # second part

print("Encrypted:", (C1, C2))


# Decryption
shared = multiply(private_key, C1)   # shared secret
original = C2 - shared[0]            # recover message

print("Decrypted Message:", original)
```

OUTPUT:

```
C:\Users\hashi\PycharmProjects\PythonProject\.venv\Scripts\python.exe
Private Key: 2
Public Key: (6, 3)
Encrypted: ((6, 3), 8)
Decrypted Message: 5

Process finished with exit code 0
```

LINE BY LINE EXPLANATION:

Curve Setup

p = 17
a = 2
b = 2

These define the curve
All calculations happen mod 17

Point Addition

def add(P, Q):

Adds two points

- If one point is empty → return the other
- If both points are same → use doubling
- If x values are same → return None (not possible)
- Otherwise:
    - Find slope
    - Calculate new point

Result = new point

Point Doubling

def double(P):

Used when same point is added

- If y = 0 → not possible
- Otherwise:
    - Calculate slope
    - Find new point

Multiplication

def multiply(k, P):

Repeat adding point P, k times

Used to generate keys

Keys

G = (5, 1)
private_key = 2
public_key = multiply(private_key, G)

G = base point
Private key = secret number
Public key = generated using G

Encryption

message = 5
k = 2

message = data
k = random number

C1 = multiply(k, G)
C2 = message + multiply(k, public_key)[0]

C1 = first part
C2 = second part

Decryption

shared = multiply(private_key, C1)
original = C2 - shared[0]

Find shared secret
Subtract it → get original message

Security Analysis

Strong Encryption

- Based on Elliptic Curve Mathematics
- Very hard to solve mathematically

Secure against brute force attacks

Small Key Size

- ECC uses small keys but strong security
- Example:
    - ECC 256-bit = RSA 3072-bit

Faster and efficient

Key Confidentiality

- Private key is never shared
- Public key is used openly

Protects secret information

Secure Key Exchange

- Shared secret is created using:

private_key × public_key

Same result on both sides
Used in encryption

Attack Resistance

- Based on Elliptic Curve Discrete Log Problem (ECDLP)
  Very difficult to solve

Weakness (Important)

- We used small numbers ($p = 17$)
  Not secure in real systems

Real systems use:

- Very large prime numbers